

Laboratorio

Extracción de
características, Bolsa
de Palabras Visuales,
Detección de
Movimiento y
Seguimiento de
Objetos

Visión por Ordenador I
Ingeniería Matemática

Universidad Pontificia Comillas ICAI



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

Laboratorio

Extracción de características, Bolsa de Palabras Visuales, Detección de Movimiento y Seguimiento de Objetos

Profesor:

Lionel Güitta
Daniel Pinilla
Luis Arias
José María Bengochea
Mario Triviño
Erik Velasco

Email

lglopez@icai.comillas.edu
dpinilla@icai.comillas.edu
learias@icai.comillas.edu
jmbengochea@icai.comillas.edu
mtrivino@comillas.edu
evelasco@icai.comillas.edu

Cover:

Tiny satellites, also known as CubeSats, are pictured after being deployed into Earth orbit from a small satellite orbital deployer on the outside of the International Space Station's Kibo laboratory module. Image Credit: NASA/Tracy Dyson



COMILLAS
UNIVERSIDAD PONTIFICIA

ICAI

Contents

1 Sesión 4: Extracción de características, Bolsa de Palabras Visuales, Detección de Movimiento y Seguimiento de Objetos	1
1.1 Materiales	1
1.2 Apartados de la práctica	1
1.3 Observaciones	1
1.4 Qué va a aprender	2
1.5 Evaluación	2
2 Apartado A: Detección de líneas rectas	3
Tarea A.1: Aplicación de Canny	3
Tarea A.2: Visualización de líneas	3
Tarea A.3: Aplicación de transformada de Hough	3
Preguntas	4
3 Apartado B: Detección de puntos de interés y Bolsa de palabras	5
B.1: Detección de puntos de interés	5
Tarea B.1.1: Filtro gaussiano con OpenCV	5
Tarea B.1.2: Generación de espacio de escalas con imágenes gaussianas	6
Tarea B.1.3: Generación de diferencias de gaussianas	6
Tarea B.1.4: Evaluación de extremos	6
Tarea B.1.5: Localización de puntos de interés y orientación de los mismos	7
Tarea B.1.6: <i>Pipeline</i> de generación de puntos de interés y descriptores	7
Tarea B.1.7: Correspondencia de características entre imágenes	8
Preguntas	8
B.2: Bolsa de palabras	9
Tarea B.2.1: Carga de <i>datasets</i> de entrenamiento y validación	9
Tarea B.2.2: Extracción de los descriptores	9
Tarea B.2.3: Creación del vocabulario	9
Tarea B.2.4: Entrenamiento del clasificador	9
Tarea B.2.5: Inferencia en el dataset de entrenamiento	9
Tarea B.2.6: Inferencia en el dataset de validación	9
Preguntas	10
4 Apartado C: Sustracción de Fondo	11
Tarea C.1: Carga de vídeo	11
Tarea C.2: Sustracción de fondo mediante diferencia de frames	11
Tarea C.3: Configuración de la Sustracción de Fondo con GMM	11
Tarea C.4: Aplicación de la Sustracción de Fondo	11
Preguntas	12
5 Apartado D: Flujo Óptico	13
Tarea D.1: Configuración del Flujo Óptico	13
Tarea D.2: Detección de Puntos de Interés	13
Tarea D.3: Cálculo y Visualización del Flujo Óptico	13
Preguntas	13
6 Apartado E: Filtro de Kalman para Seguimiento de Objetos	15
Tarea E.1: Configuración del Filtro de Kalman	15
Tarea E.2: Predicción y Corrección del Estado	15
Preguntas	16

Sesión 4: Extracción de características, Bolsa de Palabras Visuales, Detección de Movimiento y Seguimiento de Objetos

1.1. Materiales

En esta práctica se trabajará con los siguientes recursos (puede encontrarlos en la sección de Moodle *Laboratorio/Sesión 4*):

- **partA.ipynb**: notebook con el código que deberá desarrollar.
- **partB.ipynb**: notebook con el código que deberá desarrollar.
- **partC.ipynb**: notebook con el código que deberá desarrollar.
- **data**: carpeta con el material para trabajar durante la práctica.

1.2. Apartados de la práctica

La Sesión 4 del laboratorio está dividida en los siguientes apartados:

- Apartado A: Detección de líneas rectas.
- Apartado B: Detección de puntos de interés y bolsa de palabras.
- Apartado C: Sustracción de Fondo.
- Apartado D: Flujo Óptico.
- Apartado E: Filtro de Kalman para Seguimiento de Objetos.

1.3. Observaciones

Aunque el guion de la práctica y los comentarios en Markdown del Notebook estén escritos en español, observe que todo aquello que aparece en las celdas de código está escrito en inglés. Es una buena práctica que todo su código esté escrito en inglés.

Aquellas partes del código que deberá completar están marcadas con la etiqueta **TODO**.

Es muy importante que trabaje consultando la documentación de OpenCV¹ para familiarizarse de cara al examen. Tenga en cuenta que en los exámenes no podrá utilizar herramientas de ayuda como Copilot.

¹Documentación de OpenCV: <https://docs.opencv.org/4.x/index.html>

1.4. Qué va a aprender

Al finalizar esta práctica, aprenderá a implementar algoritmos de detección de líneas rectas, puntos de interés y a usar de bolsa de palabras visuales. Así como a implementar algoritmos de detección de movimiento mediante sustracción de fondo y flujo óptico, y explorará el filtro de Kalman para el seguimiento de objetos.

1.5. Evaluación

La nota que obtenga en esta sesión de laboratorio será la misma que obtenga su pareja. Los apartados de la práctica serán evaluados como refleja la Tabla 1.1.

Tarea	Valor	Resultado
Pregunta A.1	1.0	
Pregunta B.1	1.5	
Pregunta B.2	1.5	
Pregunta C.1	1.0	
Pregunta C.2	1.0	
Pregunta D.1	1.0	
Pregunta D.2	1.0	
Pregunta E.1	1.0	
Pregunta E.2	1.0	
Total	10.0	

Table 1.1: Valoración de los apartados de la práctica.

Apartado A: **Detección de líneas rectas**

Tarea A.1: Aplicación de Canny

Para obtener los bordes de las imágenes, aplique el método `cv2.Canny()` ¹ de OpenCV a las imágenes de trabajo ajustando los hiperparámetros.

Recuerde que para aplicar Canny, primero deberá pasar su imagen a escala de grises con el método `cvtColor()`. Por otro lado, observe que el método Canny devuelve una imagen binaria (blanco y negro) con los bordes detectados.

Tarea A.2: Visualización de líneas

Implemente la función `draw_lines()` para pintar las líneas detectadas sobre las imágenes. Para ello, puede utilizar el método `cv2.line()` ² de OpenCV.

Tarea A.3: Aplicación de transformada de Hough

Aplique el método `cv2.HoughLinesP()` ³ de OpenCV a las imágenes de trabajo y afine los hiperparámetros para extraer las líneas de interés. La Figura 2.1 muestra el resultado esperado al aplicar la transformada de Hough sobre una de las imágenes.



Figure 2.1: Ejemplo de detección de líneas con el método Hough.

¹Documentación del método `cv2.Canny()` en OpenCV:
https://docs.opencv.org/3.4/dd/d1a/group__imgproc__feature.html#ga04723e007ed88ddf11d9ba04e2232de

²Documentación del método `cv2.line()` en OpenCV:
https://docs.opencv.org/3.4/d6/d6e/group__imgproc__draw.html#ga7078a9fae8c7e7d13d24dac2520ae4a2

³Documentación del método `cv2.HoughLinesP()` en OpenCV:
https://docs.opencv.org/3.4/dd/d1a/group__imgproc__feature.html#ga8618180a5948286384e3b7ca02f6feeb

Preguntas

Pregunta A.1

Repita el procedimiento para extraer las líneas de las dos imágenes restantes.

Apartado B: **Detección de puntos de interés y Bolsa de palabras**

En este apartado deberá extraer los puntos de interés de una de las imágenes de la carpeta data. Una vez hecho, podrá comprobar la correspondencia con esa imagen alterada y la relación entre las mismas (Figura 3.1). Tras entender la funcionalidad de este método, pasará a obtener una bolsa de palabras a partir de un *dataset* de entrenamiento y generar predicciones para un *dataset* de evaluación.

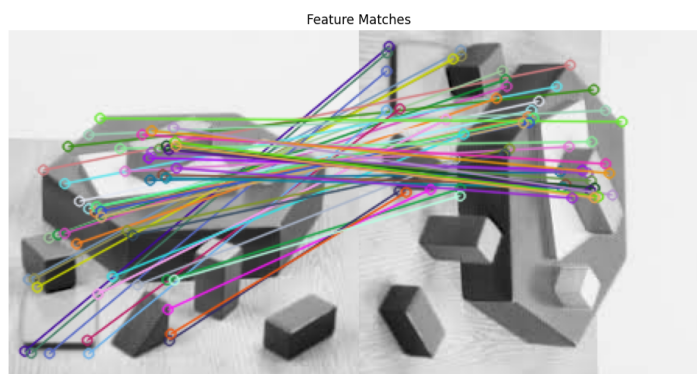


Figure 3.1: Correspondencia de características entre una imagen y la misma alterada con OpenCV.

B.1: Detección de puntos de interés

En esta sección se generarán las funciones necesarias que cubran la detección de puntos de interés y los descriptores para las imágenes empleando la transformación SIFT. La transformación SIFT consta de cuatro pasos principales:

1. Detección de extremos en espacio de escalas.
2. Localización de keypoints.
3. Asignación de orientaciones.
4. Generación de los descriptores de keypoints.

Cada uno se irá desarrollando con las funciones necesarias para completarlos. De esta forma se comenzará generando el espacio de escalas en el que se evaluarán los extremos.

Tarea B.1.1: Filtro gaussiano con OpenCV

Para generar el espacio de escalas que se desea analizar, se debe aplicar un filtrado gaussiano a la imagen original hasta obtener un número de imágenes gaussianas relevante. Para ello, debe desarrollar

la función `generateGaussianImages()` empleando el método `cv2.GaussianBlur()` ¹ de la librería OpenCV.

Como entrada, se recibe una imagen, y una lista de sigmas que deberá aplicar en el filtrado. Deberá generar tantas imágenes gaussianas como sigmas haya en la lista aplicando el filtro al resultado del filtrado anterior o a la imagen original en primera instancia.

Tarea B.1.2: Generación de espacio de escalas con imágenes gaussianas

Deberá cargar la imagen base de la carpeta de data con `cv2` y en escala de grises sin emplear la función `cv2.cvtColor()`. A continuación, se definen los parámetros iniciales del espacio de escalas, la *sigma* inicial y el número de diferencias gaussianas que se desea generar. Cabe destacar que aunque el mínimo de gaussianas necesarias son 3 para detectar los máximos y mínimos en las ventanas de 3x3x3 como se verá más adelante, es posible que no sea suficiente este valor para detectar esos puntos de interés. Por último, emplee el método `generateGaussianImages()` que ha desarrollado en el apartado anterior.

Como puede observar en el código, se le proporciona el método `generateGaussianSigmas()` que recibe la *sigma* inicial y el número de diferencias de gaussianas que desea generar. Este método devuelve una lista de valores de *sigma* que deberá aplicar en los filtrados consecutivos para la generación de las imágenes gaussianas. Si quiere generar n diferencias de gaussianas, el método le devolverá $n+1$ valores de sigma. La Figura 3.2 muestra un ejemplo del resultado esperado de este apartado.

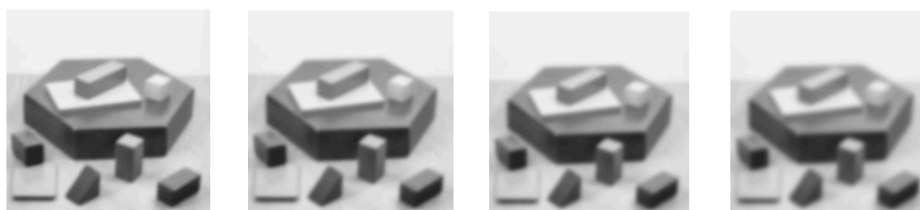


Figure 3.2: Resultados esperados de la generación de imágenes gaussianas

Tarea B.1.3: Generación de diferencias de gaussianas

A partir de la lista de imágenes anteriores, se deberá generar una nueva lista con las diferencias entre las imágenes consecutivas empleando el método `subtract` ² de la librería OpenCV. La Figura 3.3 muestra un ejemplo del resultado esperado de este apartado.

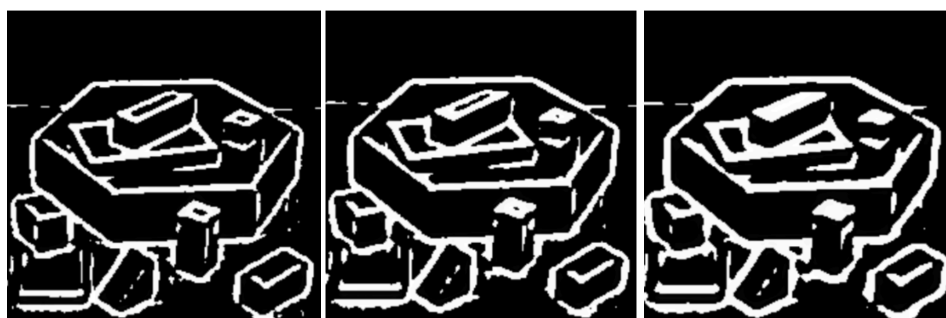


Figure 3.3: Resultados esperados de la generación de diferencias gaussianas

Tarea B.1.4: Evaluación de extremos

En esta tarea debe desarrollar la función `isPixelAnExtremum()` que evalúa si el píxel central de una ventana de 3x3x3 es un máximo o mínimo entre sus vecinos y si es superior a un umbral mínimo en

¹Documentación del método `cv2.GaussianBlur()` en OpenCV:
https://docs.opencv.org/3.4/d4/d86/group_imgproc_filter.html#gaabe8c836e97159a9193fb0b11ac52cf1

²Documentación del método `cv2.subtract()` en OpenCV:
https://docs.opencv.org/3.4/d2/de8/group_core_array.html#gaa0f00d98b4b5edeab7b8333b2de353b

valor absoluto. La ventana está compuesta por 3 regiones de 3x3 píxeles cada una que corresponden a regiones de 3 diferencias de gaussianas consecutivas. La función debe devolver *True* en caso de ser un máximo o mínimo y *False* en cualquier otro caso. **Importante:** Tras el filtrado gaussiano, los píxeles pueden tener valores negativos.

Tarea B.1.5: Localización de puntos de interés y orientación de los mismos

Empleando la función del apartado anterior deberá completar el desarrollo de la función `findScaleSpaceExtrema()`.

Primero tendrá que completar el bucle `for` para obtener en cada iteración 3 diferencias de gaussianas consecutivas y enumerarlas. Tras esto, iterar sobre las coordenadas de estas teniendo en cuenta que deberá mover la ventana de 3x3 a través de toda la imagen y no puede quedar ningún punto de esta ventana fuera de la misma. Complete la llamada a la función `isPixelAnExtremum()` con los argumentos pertinentes para evaluar si el pixel en las coordenadas de esa iteración es un punto extremo.

Si los píxeles son máximos o mínimos, se tiene una localización aproximada de los puntos de interés de esta imagen. La función `localizeExtremumViaQuadraticFit()` (ya proporcionada) refina esta localización teniendo en cuenta el gradiente y el hessiano; descartando los valores que no tengan el suficiente contraste. Con la localización final se obtiene la orientación del punto de interés gracias a la función `computeKeypointsWithOrientations()` (de nuevo, proporcionada). Del resultado de esta función se tiene el punto de interés definido completamente.

Se recomienda el uso de la función de borrado de duplicados (`removeDuplicateKeypoints()`, también proporcionada) ya que existen puntos de interés que se repiten en el resultado del proceso anterior. Se proporciona al alumno la función `visualizeKp()` que le permitirá visualizar los puntos de interés en la imagen original. La Figura 3.4 muestra el resultado que debería obtener.

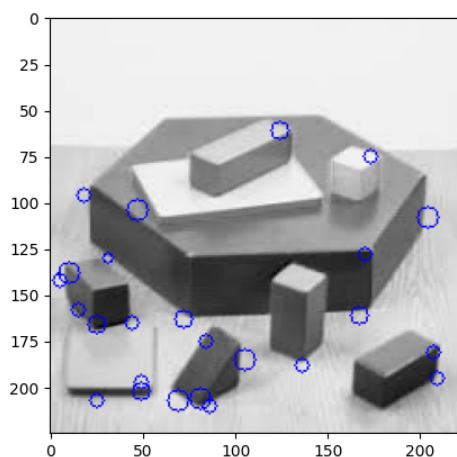


Figure 3.4: Resultados esperados de la detección de puntos de interés y orientación

Una vez obtenidas las imágenes gaussianas y los puntos de interés con sus orientaciones, se pueden generar los descriptores de las regiones vecinas a estos, que son invariantes a los cambios en la iluminación y el punto de vista 3D. En esta sesión, esto se proporciona al alumno con la función `generateDescriptors()`.

Tarea B.1.6: Pipeline de generación de puntos de interés y descriptores

En esta tarea el alumno deberá completar la función `computeKeypointsAndDescriptors()`. Esta función recibe la imagen original, el valor inicial de *sigma* y el número de diferencias de gaussianas que se desee generar. La tarea del alumno es completar la función con los métodos desarrollados en apartados anteriores para obtener el *pipeline* completo de localización de puntos de interés y generación de descriptores.

Tarea B.1.7: Correspondencia de características entre imágenes

En esta tarea, el alumno comprobará la capacidad de sistema de localizar las mismas características en una imagen alterada para luego establecer la correspondencia entre las características de la imagen original y la de la alterada. Esto se conseguirá cargando la imagen original y la alterada en escala de grises con la librería cv2 de OpenCV. Después tendrá que hacer las llamadas pertinentes al método desarrollado en el apartado anterior para obtener los puntos de interés y descriptores de ambas imágenes.

Con estos datos, empleando la función `matchFeatures()` comprobará la capacidad de establecer la correspondencia entre los puntos de interés de ambas imágenes. La Figura 3.5 muestra los resultados de la correspondencia de características entre imágenes transformadas.

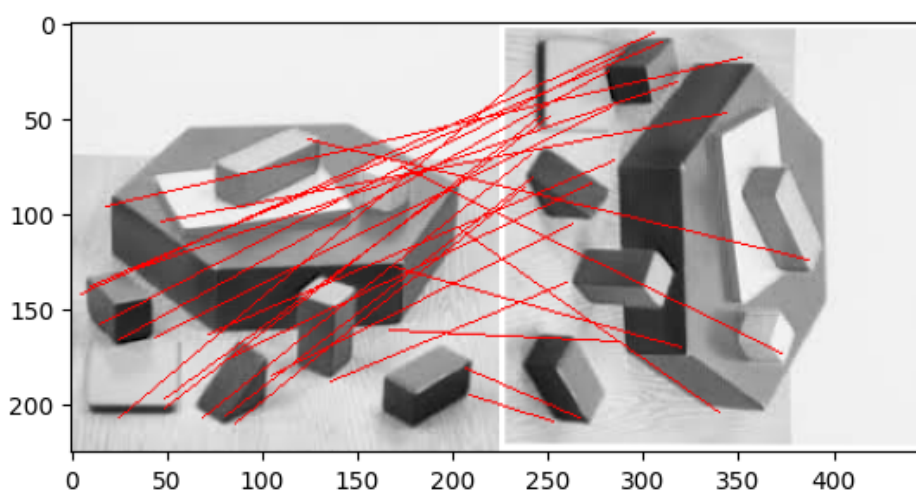


Figure 3.5: Resultados esperados de la correspondencia de características

Una vez comprendido el funcionamiento de un detector de puntos de interés y descriptores se anima al alumno a emplear el método propio de la librería OpenCV como se proporciona en el cuaderno.

Preguntas

Pregunta C.1: Correspondencia de imágenes propias y evaluación

Con los métodos desarrollados para la extracción de características, haga uso del *pipeline* que ha desarrollado y del método propio de OpenCV con sus propias imágenes. Adjunte los resultados en ambos casos y compárelos extrayendo sus propias conclusiones.

B.2: Bolsa de palabras

En este apartado se va a generar una bolsa de palabras a partir de un dataset (descargar de Moodle) de entrenamiento y se evaluará con un dataset de evaluación. Para generar la bolsa de palabras, se seguirán los pasos descritos en las sesiones de teoría:

1. Extracción de descriptores de puntos de interés de las imágenes del dataset.
2. Construcción de diccionario de palabras visuales (K-Means).
3. Para cada imagen del dataset, generar histograma de ocurrencias de palabras del diccionario.
4. Obtener un clasificador utilizando los histogramas de las imágenes del dataset (SVM).

De esta forma, se comenzará con la carga de las imágenes que compondrán los distintos *datasets* para ser procesadas posteriormente.

Tarea B.2.1: Carga de datasets de entrenamiento y validación

Como ayuda, se le proporciona una clase de Python llamada *Dataset*. Empleando el método `load()` de esta clase, cargue los *datasets* de entrenamiento y validación.

Tarea B.2.2: Extracción de los descriptores

Una vez cargados los *datasets* se puede proceder a extraer los descriptores de las imágenes que poblarán nuestra bolsa de palabras. Estos descriptores conformarán las ocurrencias de cada característica visual en la bolsa de palabras. Para ello, primero deberá cargar las imágenes del *dataset* de entrenamiento con el path de cada una de ellas y hacerlo en escala de grises. Después, empleando los métodos necesarios de OpenCV que ha visto durante la sesión, genere los descriptores y guárdelos en una lista.

Tarea B.2.3: Creación del vocabulario

Con la bolsa de palabras llena con los descriptores, se puede proceder a agrupar los mismos en las distintas palabras del vocabulario que se va a generar. Para ello, se va a ejecutar el algoritmo de clusterización de K-Means que agrupará estos descriptores. En esta tarea debe añadir los descriptores de la lista que obtuvo como resultado en la tarea anterior a la bolsa de palabras *words* que se inicializa en el código y sobre la que ejecutaremos el algoritmo. Fíjese en que se determina el número de palabras deseadas en el vocabulario final con la variable `vocabulary_size` y que esto afectará al desempeño del clasificador más adelante.

Tarea B.2.4: Entrenamiento del clasificador

Una vez tenemos las características de las imágenes agrupadas en las palabras, se va a entrenar un clasificador que sea capaz de diferenciar entre estos grupos. Este será un modelo basado en una *Support Vector Machine* (SVM). Con este clasificador podremos recibir imágenes externas al *dataset* de entrenamiento y clasificarlas en nuestro grupo de palabras.

Para entrenar el clasificador se aporta la clase *BoW* como ayuda. Añada los argumentos necesarios a los métodos especificados en la celda de código. Compruebe qué argumentos tiene que añadir accediendo al código de la clase proporcionada y analizando los métodos declarados. El modelo final se guardará en el ordenador para su uso más adelante.

Tarea B.2.5: Inferencia en el dataset de entrenamiento

Con el modelo entrenado, es hora de ver cómo trabaja y comprobar su desempeño. En este caso la métrica empleada será la precisión y podrá observar la matriz de confusión generada para las predicciones.

Al igual que en la tarea anterior, añada los argumentos necesarios a los métodos especificados en la celda de código.

Tarea B.2.6: Inferencia en el dataset de validación

Después de extraer los resultados con el dataset de entrenamiento, se tienen que comparar los mismos con los del dataset de validación donde se debería obtener un desempeño inferior.

Al igual que en la tarea anterior, añada los argumentos necesarios.

Preguntas

Pregunta B.2: En busca de los mejores parámetros

Al igual que ha iterado en la pregunta anterior con el número de palabras, itere con el resto de parámetros como el número de iteraciones para el K-Means, el número de iteraciones para el SVM, el extractor de características... Proporcione el código final con los parámetros de entrenamiento y los resultados obtenidos.

Apartado C: **Sustracción de Fondo**

Tarea C.1: Carga de vídeo

Cargue el vídeo ('visiontraffic.avi') en el cual se detectarán objetos en movimiento. Para ello, realice complete el método `read_video` que podrá reutilizar en el resto de tareas.

Tarea C.2: Sustracción de fondo mediante diferencia de frames

Realice una sustracción de fondo mediante diferencia de frames, para ello guarde un frame con el fondo estático y úselo como frame de referencia de fondo.

En primer lugar vamos a iterar sobre la lista de frames para mostrar por pantalla frame a frame e iremos avanzando con pulsando la tecla `n`, para seleccionar el frame de referencia deberá pulsar la tecla `s`.

Posteriormente se realizará la diferencia de frames entre el frame referencia y el frame actual.

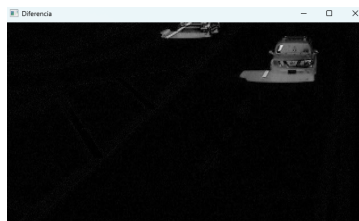


Figure 4.1: Ejemplo de resultado con diferencia de frames.

Tarea C.3: Configuración de la Sustracción de Fondo con GMM

Configure el sustractor de fondo usando el modelo de mezcla de gaussianas adaptativas (MOG2). Para ello, usaremos `cv2.createBackgroundSubtractorMOG2`¹

Tarea C.4: Aplicación de la Sustracción de Fondo

Aplice la sustracción de fondo en cada frame para extraer los objetos en movimiento.

Visualice el resultado de la detección de movimiento y guarde el vídeo con las imagenes binarias resultantes en la carpeta `data/results`. Utilize el método `cv2.VideoWriter`²

¹Documentación del método MOG2 en OpenCV:
https://docs.opencv.org/4.8.0/d7/d7b/classcv_1_1BackgroundSubtractorMOG2.html

²Documentación del método MOG2 en OpenCV:
https://docs.opencv.org/4.8.0/dd/d9e/classcv_1_1VideoWriter.html

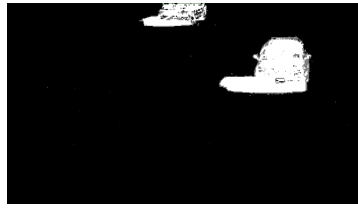


Figure 4.2: Ejemplo de resultado usando MOG2.

Preguntas

Pregunta C.1

¿Cómo afecta la variable `varThreshold` a la precisión de la detección?

Pregunta C.2

¿Qué ventajas presenta `createBackgroundSubtractorMOG2` frente a métodos simples de diferencia de imágenes?

5

Apartado D: **Flujo Óptico**

Tarea D.1: Configuración del Flujo Óptico

Para esta tarea vamos a utilizar el video ('slow_traffic_small.mp4'), para cargarlo utilice el método que se ha desarrollado anteriormente. Implemente el flujo óptico de Lucas-Kanade, útil para detectar el movimiento pixel a pixel. Para ello vamos a comenzar por definir los parametros necesarios para utilizar el método `cv2.calcOpticalFlowPyrLK`¹

Tarea D.2: Detección de Puntos de Interés

Configure los parámetros para obtener los puntos de interés iniciales en el primer frame con el método de `cv2.goodFeaturesToTrack`².

Tarea D.3: Cálculo y Visualización del Flujo Óptico

Calcule el flujo óptico con el método de Lucas-Kanade aplicando `cv2.calcOpticalFlowPyrLK` y visualice el movimiento de cada punto de interés en los siguientes frames.

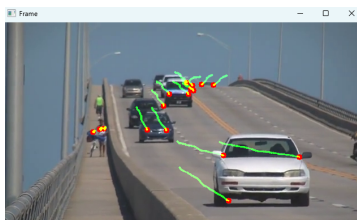


Figure 5.1: Ejemplo de resultado de obtener el flujo óptico con el metodo de Lucas-Kanade.

Preguntas

Pregunta D.1

¿Qué efecto tiene el parámetro `winSize` en la precisión del flujo óptico?

¹Documentación del método Optical Flow Lucas-Kanade en OpenCV:
https://docs.opencv.org/4.8.0/dc/d6b/group__video__track.html#ga473e4b886d0bcc6b65831eb88ed93323

²Documentación del método Optical Flow Lucas-Kanade en OpenCV:
https://docs.opencv.org/4.8.0/dd/d1a/group__imgproc__feature.html#ga1d6bb77486c8f92d79c8793ad995d541

Pregunta D.2

¿Cómo influye el parámetro `qualityLevel` en la función `cv2.goodFeaturesToTrack` al detectar puntos de interés?

Apartado E: **Filtro de Kalman para Seguimiento de Objetos**

Tarea E.1: Configuración del Filtro de Kalman

Para esta tarea vamos a utilizar el video ('slow_traffic_small.mp4'), para cargarlo utilice el método que se ha desarrollado anteriormente. Inicialice el filtro de Kalman, para ello haga uso del método `cv2.KalmanFilter`¹ de OpenCV, con una matriz de medición y transición adecuada para un seguimiento en dos dimensiones.

Complete el código del bucle que nos permitirá seleccionar el objeto (`cv2.selectROI`)² que vamos a seguir. Convierta la región de interés de la imagen a HSV y calcule el histograma `cv2.calcHist`³ con para el canal con la información de tono (*Hue*).

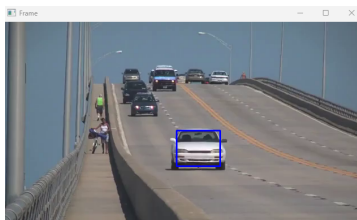


Figure 6.1: Ejemplo de ROI.

Tarea E.2: Predicción y Corrección del Estado

Realice la predicción del estado y corrija la posición estimada en cada iteración.

¹Documentación del método `KalmanFilter` en OpenCV:
https://docs.opencv.org/4.8.0/dd/d6a/classcv_1_1KalmanFilter.html

²Documentación del método para seleccionar la ROI en OpenCV:
https://docs.opencv.org/4.8.0/d7/dfc/group__highgui.html#gaabb58bae304674b429d9102a7155a86e

³Documentación del método para calculo de histograma en OpenCV:
https://docs.opencv.org/4.8.0/d6/dc7/group__imgproc__hist.html#ga4b2b5fd75503ff9e6844cc4dcdaed35d

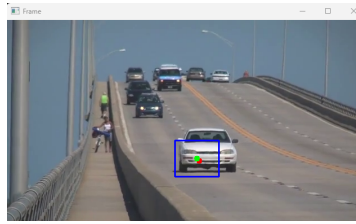


Figure 6.2: Ejemplo de seguimiento con filtro de Kalman.

Preguntas

Pregunta E.1

¿Cómo afecta el valor de `transitionMatrix` a la predicción en el filtro de Kalman?

Pregunta E.2

¿Cuál es la diferencia entre `measurementMatrix` y `transitionMatrix` en el contexto del seguimiento de objetos?